

## **Task 2- Microservice Decomposition & Bounded Contexts**

### **Step 1**

#### **> Domain**

A Domain is the complete business area of an application.

Example: For an e-commerce application, the domain is Online Shopping.

#### **> Sub Domain**

A Subdomain is a smaller part of the main domain.

Example:

- Customer Management
- Product Management
- Order Management
- Payment Management

Each of these handles a different part of the online shopping business.

#### **> Bounded Context**

A Bounded Context defines a clear area where a particular business model and rules are used.

**Example:**

- The Order context handles creating and managing orders.
- The Payment context handles payment processing.
- The Inventory context handles product stock.

Each context has its own responsibility and business rules.

#### **> Business Capability**

Business Capability is something the business needs to do.

**Examples:**

- Manage users
- Sell products
- Process orders

- Accept payments
- Manage inventory

These capabilities help us identify possible microservices.

## > **Service Boundary**

A Service Boundary clearly defines what a microservice should do and should not do.

## > **Order Service**

- Creates orders
- Updates order status
- Cancels orders

## **Step -2**

Select a Sample Business System

**Selected System:**

### **Ride Booking System**

A ride booking system allows customers to book a ride, get a driver, track the vehicle, make payments, and receive notifications.

### **Main Functions of a Ride Booking System**

1. **User Management** – Customer registration, login, and profile.
2. **Driver Management** – Driver registration and availability.
3. **Ride Management** – Book, start, complete, or cancel rides.
4. **Location Management** – Pickup, destination, and live tracking.
5. **Payment Management** – Calculate fares and process payments.
6. **Notification Management** – Send ride updates and alerts.

## **Simple Flow**

**Customer → Book Ride → Find Driver → Start Ride → Complete Ride → Make Payment → Receive Notification**

This is the business system you can use for all the next steps of your **Microservice Decomposition & Bounded Contexts** assignment.

## Step – 3

### List all business modules

First, identify all the main parts of the application.

For a **Ride Booking application**, the modules

are:

- **User** – Manages customer registration, login, and profile.
- **Driver** – Manages driver details and availability.
- **Ride** – Handles of ride booking, ride status, and trip details.
- **Payment** – Handles of fare calculation and payment processing.
- **Notification** – Sends booking, payment, and ride updates.
- **Location** – Tracks the driver's and customer's locations.

## Step 4

### Identify which modules belong together

Now, check which modules have closely related responsibilities. For example:

**User Module + Profile Management** → belong together because both manage customer information.

**Ride + Location** → are closely connected because location is required for booking and tracking a ride.

**Payment** → should remain separate because it has its own business rules and security requirements.

## Step 5

### Create service boundaries

Finally, divide the application into independent microservices with clear responsibilities.

## Possible Microservices:

### User Service

Handles user registration, login, and profiles.

### Driver Service

Handles driver registration, details, and availability.

### Ride Service

Handles the ride booking, trip management, and ride status.

### Payment Service

Handles the fare payments and transaction status.

### Notification Service

Sends SMS, email, or app notifications.

### Location Service

Handles GPS location and real-time tracking.

## Step 6

Define responsibility for every service

Each microservice should have **one clear responsibility**. For the Ride Booking application:

- **User Service** → Manages users and profiles.
- **Driver Service** → Manages driver details and availability.
- **Ride Service** → Handles ride booking, trip status, and ride history.
- **Payment Service** → Processes payments and manages transactions.
- **Notification Service** → Sends SMS, email, or app notifications.
- **Location Service** → Tracks the live location of drivers and custom

## Step 7

### Define the service details

For each microservice, define:

#### > Input

What information does the service receive?

##### **Example – Payment Service:**

Receives ride ID, user ID, and payment amount.

#### > Output

What information does the service return?

##### **Example:**

Returns **Payment Successful** or **Payment Failed**.

#### > Database

Each microservice should ideally manage **its own database**.

##### **Example:**

- User Service → User Database
- Ride Service → Ride Database
- Payment Service → Payment Database

#### > APIs

Services communicate with each other using APIs.

##### **Example:**

The Ride Service can send ride details to the Payment Service through an API.

#### > Dependencies

Identify which services depend on other services.

##### **Example flow:**

**User → Ride Service → Payment Service → Notification Service**

The Notification Service sends a message after payment or ride status changes.

## Step 8

Creation of final architecture diagram

Simple Architecture

User / Mobile App



API Gateway



User Service | Driver Service | Ride Service | Payment Service | Notification Service |  
Location Service

Each service has:

**Service → API → Own Database**

### Main Goal

Create an architecture where every microservice has:

- A clear responsibility
- Defined input and output
- Its own database

RIDE BOOKING – MICROSERVICES ARCHITECTURE

